

satsim — código explicado

Simulador de posicionamento de satélites

Documento vivo · iniciado em 02/10/2026 · branch `semana-1`

Sobre este documento. Este arquivo contém a explicação detalhada de alguns scripts do projeto satsim. Ele complementa as docstrings e os comentários do código: lá fica o *o quê* de cada função; aqui fica o *porquê* das decisões, os conceitos de Python e de mecânica orbital envolvidos e como cada peça se encaixa no restante do simulador. As explicações são acrescentadas à medida que os scripts são revisados, uma seção por script.

Convenções gerais (unidades SI, ângulos em radianos, tempo em UTC, referencial ECI) estão no ADR `docs/adr/0001-arquitetura.md`.

Scripts explicados

1. `src/satsim/propagators/base.py` — interface base dos propagadores
2. `src/satsim/astrotime.py` — tempo astronômico: UTC, Data Juliana e GMST (inclui ECI e ECEF)

1. base.py — a interface dos propagadores

`src/satsim/propagators/base.py` · card S1-05

O `base.py` não calcula nada: ele define o **contrato** que todo propagador do projeto precisa cumprir. O conteúdo é curto:

```
class Propagator(ABC):
    """Propagador de órbita: estado ECI em função do tempo desde a época."""

    @property
    @abstractmethod
    def epoch(self) -> datetime:
        """Época do propagador (UTC, tz-aware): o instante t = 0."""

    @abstractmethod
    def propagate(self, t: ArrayLike) -> tuple[NDArray[np.float64], NDArray[np.float64]]:
        """t: segundos desde ``epoch``, shape () ou (N,).

        Retorna (r, v) em ECI, SI [m, m/s], sempre com shape (N, 3).
        """

def as_time_array(t: ArrayLike) -> NDArray[np.float64]:
    arr = np.atleast_1d(np.asarray(t, dtype=float))
    if arr.ndim != 1:
        raise ValueError(f"t deve ser escalar ou 1D, recebido shape {arr.shape}")
    return arr
```

A classe abstrata Propagator(ABC)

ABC (*Abstract Base Class*, do módulo `abc` da biblioteca padrão) é uma classe que **não pode ser instanciada diretamente**. Ela funciona como um molde: declara *quais* métodos as subclasses devem ter, sem dizer *como* eles funcionam.

No satsim, ela materializa a regra central do ADR: **todo propagador implementa `propagate(t) → (r, v)`**. Ao longo do projeto haverá vários propagadores:

Propagador	Semana	Como calcula
KeplerPropagator	1	dois corpos, analítico
J2 médio	2	elementos com deriva secular
Cowell	3	integração numérica com J2 e arrasto
SGP4	3	a partir de TLE (ISS real)

O resto do código (conversão ECI → ECEF, visibilidade, constelação, exportação para a GUI) depende só de `Propagator`, nunca de um propagador específico. Isso garante duas coisas:

- **Troca sem reescrita:** `Satellite(propagator=KeplerPropagator(...))` vira `Satellite(propagator=CowellPropagator(...))` e nada mais muda.
- **Erro cedo:** se alguém criar um propagador e esquecer `propagate` ou `epoch`, o Python recusa a criação do objeto, em vez de falhar no meio de uma simulação:

```
TypeError: Can't instantiate abstract class MeuProp without an implementation for abstract method 'propagate'
```

Por isso os métodos na ABC só têm docstring, sem implementação: a docstring é a **especificação do contrato** (unidades, referencial e shapes).

O decorator @abstractmethod

Marca o método como **obrigatório**. No momento de instanciar, a ABC verifica se a subclasse sobrescreveu todos os métodos marcados. Sem esse decorator, a ABC não impediria nada: uma subclasse incompleta funcionaria até a primeira chamada e só então falharia.

O decorator @property

Transforma um método em um **atributo de leitura**: escreve-se `prop.epoch`, sem parênteses, em vez de `prop.epoch()`. Ele foi usado porque a época é um *dado* do propagador, não uma ação. Além disso, impede a atribuição acidental (`prop.epoch = outra_data` gera erro), o que protege a consistência: todos os `t` são medidos a partir dessa época, e mudá-la depois invalidaria os resultados.

Os dois decorators juntos em epoch

Significam que toda subclasse é obrigada a fornecer `epoch` como *property* de leitura. A ordem importa: `@property` por fora (em cima) e `@abstractmethod` por dentro. A ordem inversa não funciona corretamente no Python moderno. `propagate`, por sua vez, leva só `@abstractmethod`, porque é uma ação que recebe o argumento `t`.

A função `as_time_array`

Não é decorator nem faz parte da ABC: é um *helper* que todo propagador chama no início de `propagate`. Ele normaliza qualquer entrada de tempo (`5.0`, `[1, 2]`, `np.array(...)`) para um array 1D de floats com shape `(N,)`, rejeitando entradas com mais de uma dimensão. Assim, cada propagador não precisa repetir esse tratamento, e o contrato “sempre devolve `(N, 3)`” fica fácil de cumprir.

2. astrotime.py — tempo astronômico

src/satsim/astrotime.py · card S1-06

Dois referenciais: ECI e ECEF

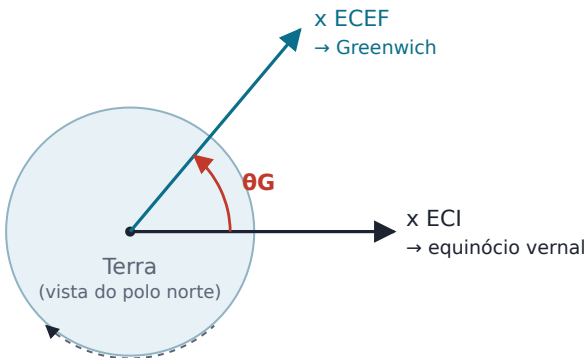
Para entender por que este script existe, é preciso distinguir os dois referenciais cartesianos do projeto. Ambos têm origem no centro da Terra e o eixo **z** ao longo do eixo de rotação, apontando para o polo norte. A diferença está em para onde aponta o eixo **x** e se ele gira ou não:

	ECI — <i>Earth-Centered Inertial</i>	ECEF — <i>Earth-Centered, Earth-Fixed</i>
Eixo x	aponta para o equinócio vernal, uma direção fixa entre as estrelas	aponta para o meridiano de Greenwich (latitude 0°, longitude 0°)
Gira?	não: os eixos ficam parados em relação às estrelas	sim: os eixos giram junto com a Terra, uma volta por dia sideral
Órbita kepleriana	elipse fixa no espaço	curva que se desloca para oeste a cada volta (é o traço no solo)
Um ponto na superfície	gira uma volta por dia	fica parado: São Bento do Sapucaí tem coordenadas constantes
Serve para	a física: a equação de movimento só tem a forma simples $\ddot{\mathbf{r}} = -\mu\mathbf{r}/r^3$ num referencial inercial; os propagadores trabalham aqui	a geografia: latitude, longitude, altitude, posição da estação de solo, elevação vista do observador e o mapa 2D da GUI

A passagem de um para o outro é uma única rotação em torno do eixo **z**, por um ângulo **θG** que cresce com o tempo:

$$\mathbf{r}_{\text{ECEF}} = \mathbf{R}_z(\theta_G) \cdot \mathbf{r}_{\text{ECI}}$$

(ADR 0001, seção Convenções)



a Terra gira para leste: θ_G cresce $\approx 360,9856^\circ$ por dia

Figura 1 — Plano do equador visto do polo norte. O eixo **z** (para fora da página) é o mesmo nos dois referenciais; o eixo **x** do ECEF acompanha Greenwich e está adiantado de θ_G em relação ao eixo **x** do ECI.

Por que o `astrotime.py` é necessário

O ângulo θ_G é o **tempo sideral médio de Greenwich (GMST)**: quanto a Terra já girou, medido a partir do equinócio vernal. Ele depende do instante absoluto. Os propagadores, porém, trabalham com t = segundos desde a época, e o usuário fornece datas no calendário civil (`datetime` em UTC). O `astrotime.py` faz a ponte entre essas três formas de expressar o tempo:

```
datetime UTC —datetime_to_jd→ JD —gmst→  $\theta_G$  [rad] → Rz( $\theta_G$ ): ECI → ECEF
               ←jd_to_datetime←
t do propagador —jd_from_epoch(epoch_jd, t)→ JD
```

Sem este módulo não seria possível saber *sobre qual ponto da Terra* o satélite está: o propagador diz onde o satélite está no espaço (ECI), mas latitude, longitude, o traço no solo do mapa 2D (card S1-08) e a visibilidade sobre São Bento do Sapucaí (semana 4) exigem o ECEF, e o ECEF exige θ_G no instante certo.

A Data Juliana (JD)

A Data Juliana conta os dias, com fração decimal, desde o meio-dia de 1º de janeiro de 4713 a.C. É a escala de tempo padrão da astronomia porque é um **único número contínuo**: sem meses de tamanhos diferentes, anos bissextos ou fusos horários, a diferença entre dois instantes é uma simples subtração. Alguns valores de referência usados nos testes:

Instante (UTC)	JD	Observação
1970-01-01 00:00	2440587,5	época do <i>Unix timestamp</i>
2000-01-01 00:00	2451544,5	o dia juliano começa ao meio-dia, daí o “,5”
2000-01-01 12:00	2451545,0	J2000.0, a época de referência da fórmula do GMST

`datetime_to_jd` usa a relação exata $JD = 2440587,5 + \text{timestamp}/86400$: o `timestamp()` do Python conta segundos desde 1970-01-01 UTC e já desconta o fuso horário, de modo que 09:00 no horário de Brasília (−03:00) e 12:00 UTC dão o mesmo JD. Um `datetime` sem fuso (*naive*) é rejeitado com `ValueError`, porque não dá para saber a qual instante ele se refere. `jd_to_datetime` faz o caminho inverso, arredondando a microssegundos; como um float com $JD \approx 2,5$ milhões tem resolução de cerca de 40 μs , a ida e volta tem erro de dezenas de microssegundos (o critério do card é $< 100 \mu\text{s}$). `jd_from_epoch` converte o t do propagador em JD: $\text{epoch_jd} + t/86400$, vetorizado.

O GMST: a fórmula IAU-82

A função `gmst` implementa a fórmula de Vallado (eq. 3-47), com T em séculos julianos desde J2000:

```
T = (JD - 2451545,0) / 36525
 $\theta$  [s] = 67310,54841 + (876600·3600 + 8640184,812866)·T + 0,093104·T2 - 6,2e-6·T3
 $\theta_G$  [rad] = ( $\theta \bmod 86400$ ) · 2 $\pi$  / 86400
```

O resultado intermediário está em *segundos de tempo sideral*: um dia inteiro (86 400 s) equivale a uma volta (2π). Cada termo tem um papel:

- **67310,54841 s** é o GMST em J2000.0, ou seja, 280,46061837° (teste `test_gmst_j2000`);
- **876600·3600·T** são as voltas inteiras: 876 600 h = 36 525 dias × 24 h, isto é, uma volta por dia solar;
- **8640184,812866·T** é o adiantamento sideral: a cada dia solar a Terra gira um pouco mais que 360°, porque também avançou na órbita em volta do Sol;
- os termos em **T²** e **T³** são correções lentas (precessão do equinócio), desprezíveis em poucos anos.

Esse “um pouco mais” é a chave do projeto: num dia o GMST avança 360° + **0,9856°** (teste `test_gmst_daily_rate`). Os 0,9856°/dia são exatamente o movimento aparente do Sol, o mesmo número que define a órbita heliossíncrona (Sem4 eq. 26): o plano da órbita precisa girar 0,9856° por dia para acompanhar o Sol.

Validação

- **Vallado, Exemplo 3-5** (20/08/1992 12:14 UT1): JD = 2448855,009722 e GMST = 152,5788°, reproduzidos.
- **Skyfield**, só nos testes: em 200 datas aleatórias entre 1990 e 2040, a diferença máxima foi de 0,006 s de tempo sideral (critério: < 0,1 s). O Skyfield usa uma formulação mais recente (IAU 2000/2006, baseada no ângulo de rotação da Terra); a diferença de milissegundos é a discrepância entre os dois modelos.

Simplificações assumidas

Simplificação	Por quê	Impacto
UTC tratado como UT1	a fórmula pede UT1, o tempo da rotação real da Terra, que exige tabelas publicadas pelo IERS; usamos UTC direto	$ UT1 - UTC < 0,9 \text{ s} = \text{até } \approx 0,4 \text{ km no equador}$
Segundos intercalares ignorados	<code>datetime</code> do Python não os representa (todo dia tem 86 400 s)	intervalos que cruzam um segundo intercalar ficam 1 s mais curtos
Taxa do GMST $\neq$ <code>OMEGA_EARTH</code>	a fórmula dá $\approx 7,2921158553\text{e-}5 \text{ rad/s}$; os PDFs usam a constante $7,292115\text{e-}5 \text{ rad/s}$	$\approx 5 \text{ m/dia no equador}$; por isso ECI → ECEF usa a fórmula completa, nunca <code>OMEGA_EARTH · t</code>

Para visibilidade de satélites, em que importam centenas de metros, essas simplificações são aceitáveis. Estão registradas no ADR e na docstring do módulo.